DSC 180A PA 5 - Issac Roy

due Sunday, November 1 at 11:59PM

Context. Our project is about quantizing Pi0.5, a robot control model with a vision encoder, a language model prefix, and an action expert, so it runs fast on consumer GPUs. This week I built a roofline model, which estimates the fastest each stage could run from how much math it does and how much memory it reads, to see where quantization can actually help.

Problem 1.

Describe one result you found interesting doing your task. Describe what it is in your own words and why you found it interesting.

Solution: What I found most interesting is that the different stages of the model are bottlenecked by completely different things. The vision encoder (SigLIP) and the language model prefix both process hundreds of tokens at once, so they end up compute-bound. The action expert only works on 50 action tokens per step, but it runs 10 times, and each time it has to read its weights and the cached prefix again, so it ends up memory-bound.

This was interesting because mixed precision always seemed like the better approach, but I wasn't sure how to actually decide what gets which format. Weight-only 4-bit quantization only shrinks the bytes you load, so it does basically nothing for the compute-bound stages, while 8-bit weights and activations actually speed up the matmuls themselves. So the roofline kind of clears it up. The memory-bound action expert wants smaller weights, and the compute-bound stages want lower-precision math.

Problem 2.

Describe one thing your struggled with doing your task. Try to be specific about what is causing you difficulty and why.

Solution: The thing I struggled with most was pinning down the actual shapes going into the model, specifically how many text tokens are in the prompt. Pi0.5 writes the robot's state into the prompt as text, so the prompt length isn't fixed. Physical Intelligence's openpi config and the LeRobot checkpoint we're using don't agree either. One turns the state off for LIBERO entirely, and the other allows up to 200 prompt tokens, while I had estimated 48. This matters since the LLM prefix is compute-bound and its cost scales with the number of tokens.

Problem 3.

In your work, identify one choice that you made in your investigation. Why did you make this choice? Were there other choices you could've made that would be reasonable? For example, this may be a choice in sub-population to study, or a choice in parameter for a model.

Solution: The main choice I made was to build a roofline model before running anything on a GPU. It just counts the FLOPs and bytes for each stage from the model's shapes and compares them against the GPU's peak compute and memory bandwidth. I did this because it's cheap, it doesn't need any cluster time, and it tells us ahead of time where quantization can even help and what the best possible speedup would be for each format.

Another option would have been to just profile the BF16 model directly on a 3090 Ti or on NRP. That would give real numbers, including stuff the roofline ignores like kernel launch overhead. But without a bound to compare against, we can't tell how far a measurement sits from optimal.